

Towards Bangla Mental Health Detection: A Novel Annotated Dataset and Frequency-Based Vectorization

Asfia Moon Oishy

Department of Computer Science and Engineering

Uttara University

Dhaka, Bangladesh

asfiamoon126@gmail.com

Abstract—Mental health disorders have a significant impact on thoughts, feelings, and behavior. Early detection of these disorders is essential for enhancing quality of life. However, research on low-resource languages such as Bangla is still limited, despite the potential of Natural Language Processing (NLP) and Machine Learning (ML) for detecting mental illnesses. The two main contributions of this work are (1) a new annotated dataset of 4,207 Bangla social media texts with expert validation and strict annotation protocols, and (2) a custom frequency-based vectorizer intended to identify class-specific linguistic patterns in mental health text. With 93.41% accuracy using Logistic Regression, the suggested vectorizer outperforms TF-IDF by 15.77% and Count Vectorizer by 22.24%. It explicitly models positive and negative word frequency patterns. Our dataset and vectorization technique show the potential for affordable, easily accessible screening solutions and lay the groundwork for Bangla mental health informatics.

Index Terms—Bangla Text, Mental Illness Detection, NLP, Machine Learning, Sentiment Analysis, Mental Health

I. INTRODUCTION

Mental illness significantly impacts mood, thinking, and behavior, yet millions remain untreated due to stigma, restricted access to care, and lack of early detection [1]–[3]. This public health problem is made worse in Bangladesh by social taboos and a lack of diagnostic resources. Social media linguistic cues can be used to express emotions and disclose underlying mental states. While Natural Language Processing (NLP) and Machine Learning (ML) have successfully detected [4] such cues in high-resource languages like English, Bangla, spoken by over 300 million people, remains significantly underexplored due to its complex morphology and limited computational resources.

This study addresses a significant gap in the field through two main contributions: (1) a rigorously annotated dataset consisting of 4,207 Bangla social media texts, which demonstrates substantial inter-annotator agreement ($\kappa = 0.78$) and has been validated by experts; and (2) a custom frequency-based vectorizer that specifically models linguistic patterns related to different classes. This vectorizer achieved an accuracy of 93.41% using Logistic Regression, outperforming the TF-IDF method by 15.77% and the Count Vectorizer by 22.24%. Our dataset was collected from Facebook, YouTube, and blogs,

with binary classification (normal vs. sick) performed using standard classifiers after preprocessing with stop-word removal, stemming, and noise elimination.

A. Contributions

- 1) **Bangla Mental Health Dataset:** First rigorously annotated corpus of 4,207 texts with documented protocols and quality metrics.
- 2) **Custom Frequency-Based Vectorizer:** Domain-specific method outperformed standard vectorization by 15–22%.
- 3) **Bangla-Specific NLP Pipeline:** Complete preprocessing workflow addressing code-mixing and morphological challenges.
- 4) **Comparative Evaluation:** Demonstrates lightweight models with proper feature engineering achieve high accuracy in low-resource settings.

This work provides a foundation for accessible mental health screening solutions for 300+ million Bangla speakers worldwide.

II. RELATED WORKS

The identification of mental health conditions from textual data has garnered attention due to the widespread use of social media, which offers text that reflects psychological states and enables automated detection through NLP and ML. [5].

Social Media-Based Detection: Tadisetty and Ghazinour [6] developed an NLP approach for anonymous mental illness prediction, Doan [7] extracted health indicators from Twitter, Trifan and Oliveira [8] analyzed linguistic features to detect mental and eating disorders, Chomutare [9] used text classification for depression detection, and Guntuku et al. [10] combined linguistic and socio-demographic data for psychological well-being prediction.

Clinical Text Analysis: Lin et al. [11] proposed a multi-label SVM for clinical disorder identification, He et al. [12] applied NLP for PTSD screening, and Shickel et al. [13] detected cognitive distortions automatically.

Suicide Risk Assessment: Boag [2] predicted mental health readmissions, Adamou et al. [1] evaluated suicide risk via

data mining, Gaur et al. [14], [15] proposed a knowledge-aware intervention framework, and Boukil et al. [16] used deep learning for suicide-related sentiment analysis.

Deep Learning Approaches: Tran and Kavuluru [17] applied neural networks to psychiatric records, Ghosh and Anwar [18] estimated depression severity from social media, and Tadesse et al. [19] detected suicidal intent using deep learning.

Research Gap: Most studies focus on English datasets, with limited work on low-resource languages like Bangla. Existing Bangla studies mainly address depression or binary sentiment. This study proposes a Bangla-specific NLP pipeline with custom preprocessing, feature extraction, and evaluation of multiple ML classifiers for mental health detection from Bangla social media text.

III. METHODOLOGY

This study applies ML and NLP to detect mental illness in Bangla text using binary classification (Normal vs. Sick). A labeled dataset from blogs, YouTube comments, and social media was preprocessed with stemming, stop-word removal, and text cleaning. CountVectorizer, TF-IDF, and a custom Bangla vectorizer converted text into numerical features. Naïve Bayes and Logistic Regression were trained and evaluated using Precision, Recall, F1-score, and accuracy.

A. System Overview and Dataset Description

The proposed system employs NLP and ML techniques in two phases: (1) dataset preparation and (2) model-based classification (Fig. 1).

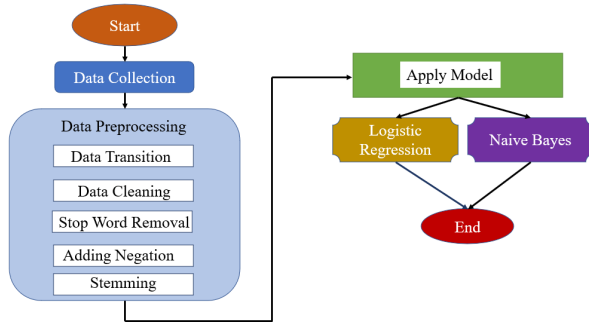


Fig. 1. Proposed system workflow.

Bangla text data were collected from social media, YouTube comments, and blogs. Each sample was manually labeled based on mental state indicators. After cleaning and filtering, **4,207 valid and unique** samples were retained from an initial 4,568 records [20]. The dataset covers Depression, Anxiety, PTSD, Bipolar Disorder, OCD, BPD, ADHD, and Alzheimer's, consolidated into binary classes: *Normal* (2,125) and *Sick* (2,124), ensuring balanced training.

B. Dataset Annotation Guidelines

The annotation process followed systematic guidelines to ensure consistency and reliability.

Annotation Framework: Texts were categorized into binary classes for computational feasibility.

Annotator Selection: Three graduate students with psychology and linguistics backgrounds were trained on linguistic markers of distress in Bangla and cultural context of mental health expression in Bangladesh.

Protocol: At least two annotators independently labeled each sample. Normal indicators included positive emotions and balanced tone; Sick indicators included persistent sadness, suicidal ideation, severe anxiety, and cognitive distortions.

Quality Assurance: Inter-annotator agreement achieved Cohen's Kappa $\kappa = 0.78$ (substantial agreement). Disagreements were resolved with a clinical psychologist, and 15% underwent double verification.

Ethical Considerations: All data was anonymized with personally identifiable information removed.

Challenges: Code-mixing (Bangla-English), cultural euphemisms, distinguishing temporary emotions from persistent conditions, and handling sarcasm.

SL	Text	Outcome
1	আমি আর কোন সাহায্যের জন্য চিৎকার করতে পারি না.. আমি জানি না আর কি করব।	Sick
2	তোমাকে এটা পড়তে হবে না আমার শুধু কথা বলার মতো কেউ নেই.. আমি এমনকি এই	Normal
3	যথেষ্ট অসুস্থ নয়। আমি বছর ধরে একটি কষ্টের সাথে লড়াই করেছি, কিন্তু আমি কখনও	Sick
4	এক্সপোজারগুলি প্রাথমিকভাবে কাঠিন্য মনে হতে পারে, তবে সেগুলি ওভারটাইম সহজ	Normal
5	আমি কি ক্ষমার যোগ্য? আমি বেশ কয়েক বছর আগে খুব বিষাক্ত ব্যক্তি ছিলাম। আমি	Sick

Fig. 2. Manually labeled Dataset

C. Data Preprocessing

Preprocessing converts raw data into a format suitable for analysis through the following steps:

Data Cleaning: Handling missing data, removing duplicates and outliers, correcting data types, standardizing inconsistent data, and removing irrelevant content (emojis).

Stop-words Removal: A predefined list of 732 Bangla stop words was used to reduce dimensionality [21].

Stemming: Words were reduced to root forms using the Bangla stemmer library [22].

Data Reduction: Text was truncated at 2,000 characters to manage computational complexity while retaining essential information.

D. Feature Extraction

Cleaned text was transformed into numerical feature vectors using three methods: **CountVectorizer** (word frequency counts), **TF-IDF Vectorizer** (importance-weighted frequencies), and a **Custom Frequency Vectorizer** designed for

SL	Text	Preprocessed	Outcome
1	আমি আর কোন সাহায্যের জন্য চিৎকার করতে পারি না.. আমি জানি	আমি সাহায্য চিৎকার করতে পারি না.. আমি জানি না আর কি	Sick
2	তোমাকে এটা পড়তে হবে না আমার শুধু কথা বলার মতো কেউ নেই..	তোমাকে পড়তে না আমার শুধু কথা বলার মতো কেউ নেই..	Normal
3	যথেষ্ট অসুস্থ নয়। আমি বছর ধরে একটি কষ্টের সাথে লড়াই করেছি	অসুস্থ নয়। আমি বছর ধরে একটি কষ্টের সাথে লড়াই করেছি	Sick
4	এক্সপোজারগুলি প্রাথমিকভাবে কঠিন মনে হতে পারে, তবে সেগুলি	এক্সপোজারগুলি কঠিন মনে হতে পারে, তবে সেগুলি	Normal
5	আমি কি ক্ষমার যোগ্য? আমি বেশ কয়েক বছর আগে খুব বিষাক্ত ব্যক্তি	আমি ক্ষমার যোগ্য? আমি বেশ বছর আগে খুব বিষাক্ত ব্যক্তি	Sick

Fig. 3. Dataset after Preprocessing

Bangla mental health vocabulary [23]. N-gram analysis (uni-gram, bi-gram, tri-gram) captured word context and sequence information.

E. Tokenization

Tokenization is the process of converting a continuous sequence of text into a sequence of tokens (terms or words). It is an essential pre-processing step in NLP as it breaks down sentences into constituent words for further processing and analysis.

Mathematical Formulation:

Let's say we have a sequence of text:

$$S = [s_1, s_2, s_3, \dots, s_n] \quad (1)$$

The goal of tokenization is to break this sequence into a list of tokens:

$$T = [t_1, t_2, t_3, \dots, t_m] \quad (2)$$

where each token t_i is a word or term in the text. The tokenization process can be represented as:

$$T = \text{Tokenize}(S) \quad (3)$$

where Tokenize is a function that takes in a sequence of text S and outputs a list of tokens T .

F. Count Vectorizer

Count Vectorizer is a method to convert text into numerical features for machine learning models in text classification tasks. The basic idea is to count the frequency of words in a document and represent each document as a vector of these word frequencies. This vector representation is called a bag-of-words representation, as it ignores word order and only considers word frequency [24].

Mathematical Representation:

Let's consider a corpus of N documents and M unique words in the corpus. The Count Vectorizer creates a matrix X of size $N \times M$, where each row represents a document and each column represents a unique word in the corpus.

The element $X_{i,j}$ represents the frequency of the word j in the document i . The Count Vectorizer is represented mathematically as follows:

$$X_{i,j} = \text{frequency of word } j \text{ in document } i \quad (4)$$

where $X_{i,j}$ is the frequency of word j in document i .

G. TF-IDF Vectorizer

The TF-IDF (term frequency-inverse document frequency) is a numerical statistic that reflects the importance of a word to a document in a corpus (collection of documents). The basic idea is to weight words that are more frequently used in a document but less frequently used in the entire corpus more heavily [25].

Mathematical Definition:

The TF-IDF for a term t in a document d in a corpus D can be defined as:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D) \quad (5)$$

Term Frequency (TF): Measures how frequently a term t appears in document d :

$$\text{TF}(t, d) = \frac{n_{t,d}}{\sum_{k=1}^{n_d} n_{k,d}} \quad (6)$$

where:

- $n_{t,d}$ is the number of times term t appears in document d
- n_d is the total number of terms in document d

Inverse Document Frequency (IDF): Measures the rarity of a term t in the entire corpus D :

$$\text{IDF}(t, D) = \log \left(\frac{N}{|d \in D : t \in d|} \right) \quad (7)$$

where:

- N is the total number of documents in the corpus D
- $|d \in D : t \in d|$ is the number of documents in the corpus that contain term t

The combined use of term frequency and inverse document frequency results in a score that reflects the importance of a term to a particular document within a corpus.

H. Custom Vectorizer for Frequency Count

Traditional vectorizers like TF-IDF and Count Vectorizer often fail to capture domain-specific linguistic patterns in low-resource languages like Bangla, particularly for mental health text. We developed a custom frequency-based vectorizer tailored to Bangla mental health vocabulary.

1) *Motivation:* Our custom vectorizer explicitly models positive and negative frequency patterns to better distinguish between Normal and Sick classes, capturing domain-specific terminology and sentiment patterns that generic vectorizers may miss.

2) *Architecture:* Algorithm 1 presents the complete process for our custom frequency-based vectorizer [26] [27].

This dual-frequency representation allows the classifier to learn class-specific word importance patterns.

Algorithm 1 Custom Frequency Vectorizer

```

1: Input: Corpus  $C = \{d_1, d_2, \dots, d_n\}$ , Labels  $L = \{l_1, l_2, \dots, l_n\}$ 
2: Output: Feature matrix  $F$ 
3:
4: // Build class-specific vocabularies
5: Initialize empty lists  $V_{pos}, V_{neg}$ 
6: for each document  $d_i$  in  $C$  do
7:    $words = \text{split}(d_i)$ 
8:   for each word  $w$  in  $words$  do
9:     if  $l_i == \text{"Normal"}$  AND  $w \notin V_{pos}$  then
10:      Add  $w$  to  $V_{pos}$ 
11:     else if  $l_i == \text{"Sick"}$  AND  $w \notin V_{neg}$  then
12:      Add  $w$  to  $V_{neg}$ 
13:     end if
14:   end for
15: end for
16:
17: // Extract features for each document
18: Initialize empty feature matrix  $F$ 
19: for each document  $d_i$  in  $C$  do
20:    $words = \text{split}(d_i)$ 
21:    $freq_{pos} = \sum_{w \in words} \mathbb{1}(w \in V_{pos})$ 
22:    $freq_{neg} = \sum_{w \in words} \mathbb{1}(w \in V_{neg})$ 
23:    $f_i = [1, freq_{pos}, freq_{neg}]$  {Bias term + frequencies}
24:   Append  $f_i$  to  $F$ 
25: end for
26: return  $F$ 

```

3) *Advantages over Standard Vectorizers:*

- **Class-Aware Features:** Explicitly models positive/negative class frequencies
- **Domain Specificity:** Captures mental health-specific vocabulary patterns
- **Reduced Sparsity:** Aggregated frequencies reduce feature space dimensionality

The custom vectorizer creates separate vocabularies for Normal and Sick classes, then represents each document as a feature vector $[1, freq_{pos}, freq_{neg}]$, where $freq_{pos}$ and $freq_{neg}$ count word occurrences in their respective class vocabularies. This dual-frequency encoding provides explicit class-discriminative features that significantly improve classification performance.

I. Classification Models

The classification phase employed two supervised ML algorithms: Logistic Regression (LR) and Naive Bayes (NB). Both models were trained on the vectorized dataset to classify text into *Normal* or *Sick* categories.

1) **Logistic Regression (LR):** It is a linear model that is used to predict binary outcomes(yes/no, true/false) based on the input features. Logistic regression is a linear classifier used in binary text classification that predicts class probabilities by combining features with learned weights.

Mathematical Formulation:

For a dataset of N examples and K classes, the logistic regression classifier is defined as a linear combination of the features x and the weights w :

$$z = x^T w \quad (8)$$

$$\hat{y} = \text{softmax}(z) \quad (9)$$

where x is the feature vector of the input, w is the weight matrix, z is the linear combination of the feature vector and the weight matrix, and $\hat{y}$ is the predicted probability for each class.

The softmax function calculates the predicted class probabilities:

$$\text{softmax}(z_j) = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \quad (10)$$

where z_j is the j th element of z and K is the number of classes.

The goal is to maximize the likelihood of observing the correct class for each example. The weights w are optimized using maximum likelihood estimation, typically implemented through gradient descent or limited-memory BFGS.

Example Application:

Consider classifying sentences as either “Normal” or “Sick” using logistic regression. Each sentence is represented by a feature vector where features correspond to word frequencies or patterns. Table I shows an example dataset:

TABLE I
EXAMPLE DATASET FOR TEXT CLASSIFICATION

Sentence	Love	Hate	Good	...	Label
I have suicidal thoughts	0	0	0	...	0
Do not feel like living	0	0	0	...	0
Love this life	1	0	0	...	1
Hate this life	0	1	0	...	0
I feel good	0	0	1	...	1

Training Process:

- 1) Split the dataset into training(80%) and testing sets(20%)
- 2) Train the model using equations (8) and (9), where input features are word frequencies/presences and output is the binary label
- 3) Evaluate performance using metrics like accuracy, precision, recall, and F1-score

The model learns the relationship between input features (word frequencies) and output labels, assigning higher probabilities to sentences resembling the target class and classifying new sentences based on their feature representations.

2) **Naive Bayes (NB):** It is a probabilistic model that uses Bayes’ theorem to predict the class label of new instances based on the probabilities of the features given the class label.

Naive Bayes is a probabilistic algorithm based on Bayes’ theorem that assumes independence between features (words in text). It is widely used for binary text classification.

Working Process:

- 1) **Preprocessing:** Clean and preprocess text data to remove stop words, apply stemming, and eliminate irrelevant information
- 2) **Vectorization:** Convert text data into numerical vectors using methods like CountVectorizer or TfidfVectorizer
- 3) **Training:** Compute the probabilities of each class and the probabilities of each feature (word) given each class
- 4) **Prediction:** For a new sample text, convert it into a vector, calculate the probability of each class given the sample text, and assign the class with the highest probability

Mathematical Formulation:

For predicting a class C , where N is the total number of training sentences and N' is the number of sentences with that class type:

$$P(C) = \frac{N'}{N} \quad (11)$$

For each word in the test sentence, the probability of the word for a particular class is calculated. Then all probability values for a particular class are multiplied, and the class with the highest probability is chosen:

$$P(w_i|c) = \frac{\text{count}(w_i, c) + 1}{\left(\sum_{w=1}^V \text{count}(w, c)\right) + |v|} \quad (12)$$

where w_i is the i th word, c is the class, V is the vocabulary size, and $|v|$ is the total number of unique words.

Example Application:

Table II shows example sentences where features are word counts and labels indicate Normal or Sick mental states.

TABLE II
EXAMPLE DATASET FOR TEXT CLASSIFICATION

Sentence	Love	Hate	Good	...	Label
I Love myself	1	0	0	...	1
Do not feel like living	0	0	0	...	0
Love this life. It is good	1	0	1	...	1
Hate this life	0	1	0	...	0
I feel good	0	0	1	...	1

Consider the last sentence “I feel good” as the test sentence. The probability of Normal and Sick classes for each word (‘I’, ‘feel’, ‘good’) is calculated separately using equations (11) and (12). Then:

$$P('I'|Normal) \times P('feel'|Normal) \times P('good'|Normal)$$

is compared with:

$$P('I'|Sick) \times P('feel'|Sick) \times P('good'|Sick)$$

The class with the higher probability is predicted.

IV. EXPERIMENTAL RESULTS

A. Experimental Setup

Experiments were conducted on an Intel Core i5-1135G7 processor with 8GB RAM. Models were implemented in Python 3.7 using Jupyter Notebook and Google Colab. Key libraries included Pandas, NumPy, Scikit-learn, and Bengali-stemmer.

B. Dataset

The dataset consists of 4,207 Bangla texts collected from Facebook, YouTube, and blogs, labeled as Normal (2,125) and Sick (2,124). Preprocessing included cleaning, stop-word removal (732 words), stemming, and text truncation at 2,000 characters.

C. Evaluation Metrics

Model performance was measured using Accuracy, Precision, Recall, and F1 Score.

D. Results

Three vectorization methods were evaluated with Logistic Regression (LR) and Multinomial Naive Bayes (MNB). Table III and Figure 4 summarize the performance.

TABLE III
PERFORMANCE COMPARISON OF VECTORIZERS WITH LR AND MNB

Vectorizer	Metric	LR (%)	MNB (%)
TF-IDF	Accuracy	77.64	67.18
	Precision	77.54	70.64
	Recall	77.64	67.17
	F1 Score	75.95	57.92
Count	Accuracy	71.17	68.12
	Precision	72.69	71.26
	Recall	71.17	68.12
	F1 Score	71.68	68.93
Custom	Accuracy	93.41	71.88
	Precision	94.00	73.08
	Recall	93.41	71.88
	F1 Score	93.21	69.40

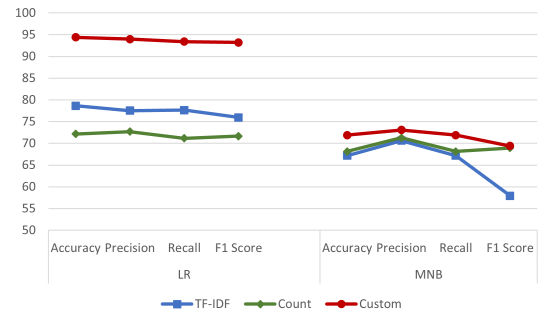


Fig. 4. Line Chart of Performance Comparison.

The proposed custom vectorizer achieves the highest performance, reaching 93.41% accuracy with Logistic Regression and outperforming TF-IDF and Count Vectorizer across all

metrics, confirming the benefit of domain-specific feature engineering.

E. Analysis and Discussion

1) *Why Custom Vectorizer Excels with Logistic Regression:* The custom vectorizer achieves 93.41% accuracy with Logistic Regression, outperforming TF-IDF (77.64%) and Count Vectorizer (71.17%) due to:

- **Class-Specific Features:** The dual-frequency encoding, which includes both positive and negative frequencies ($freq_{pos}$, $freq_{neg}$), provides clear discriminative information that aligns well with Logistic Regression's linear decision boundary.
- **Reduced Sparsity:** Aggregated frequencies reduce noise inherent in raw count-based methods.

2) *Naive Bayes Performance:* MNB achieves moderate performance (71.88%) but underperforms LR because the feature independence assumption is violated by correlated dual-frequency features, and class-aggregated frequencies create skewed probability distributions.

3) *Dataset Quality Impact:* The balanced class distribution (Normal: 2,125, Sick: 2,124) and rigorous annotation ($\kappa = 0.78$) contribute to model reliability and prevent class imbalance issues.

V. CONCLUSION

Two main contributions for Bangla mental health detection were presented in this work: a custom frequency-based vectorizer that models class-specific word patterns and a rigorously annotated dataset of 4,207 social media texts that achieved significant inter-annotator agreement ($\kappa = 0.78$). The suggested vectorizer outperformed TF-IDF by 15.77% and Count Vectorizer by 22.24% with 93.41% accuracy with Logistic Regression, proving that domain-specific feature engineering greatly improves performance for low-resource languages with lightweight classifiers. Code-mixing, cultural euphemisms, and a lack of Bangla NLP resources were major obstacles. In the future, the dataset will be expanded, BERT-based deep learning techniques will be investigated, and multi-class classification for fine-grained mental health condition detection will be extended. Our vectorization approach and publicly accessible dataset [20] lay the groundwork for affordable, easily accessible screening solutions for more than 300 million Bangla speakers worldwide.

REFERENCES

- [1] M. Adamou, "Mining free-text medical notes for suicide risk assessment," pp. 1–8, 2018.
- [2] W. Boag, "Hard for humans, hard for machines: Predicting readmission after psychiatric hospitalization using narrative notes," *Translational Psychiatry*, vol. 11, pp. 1–6, 2021.
- [3] F. Tasnim, S. U. Habiba, N. Nafisa, and A. Ahmed, "Depressive bangla text detection from social media post using different data mining techniques," in *Computational Intelligence in Machine Learning: Select Proceedings of ICCIML 2021*. Springer, 2022, pp. 237–247.
- [4] S. Saifullah, Y. Fauziah, and A. S. Aribowo, "Comparison of machine learning for sentiment analysis in detecting anxiety based on social media data," *arXiv preprint arXiv:2101.06353*, 2021.
- [5] T. Rutowski, E. Shriberg, A. Harati, Y. Lu, R. Oliveira, and P. Chlebek, "Cross-demographic portability of deep nlp-based depression models," in *2021 IEEE spoken language technology workshop (SLT)*. IEEE, 2021, pp. 1052–1057.
- [6] S. and G. T. K., "Anonymous prediction of mental illness in social media," pp. 954–960, 2021.
- [7] S. Doan, "Extracting health-related causality from twitter messages using natural language processing," *BMC Med. Inform. Decis. Mak.*, vol. 19, pp. 71–77, 2019.
- [8] A. and O. T. J.L., "Bioinfo@ uavr at erisk 2019: delving into social media texts for the early detection of mental and food disorders," *CLEF (Working Notes)*, 2019.
- [9] T. Chomutare, "Text classification to automatically identify online patients vulnerable to depression," pp. 125–130, 2014.
- [10] S. C. G. G. S., and L. Ungar, "Current and future psychological health prediction using language and socio-demographics of children for the clypsch 2018 shared task," pp. 98–106, 2018.
- [11] Wei, J. L. Dan, and Y. Lu, "Disorder recognition in clinical texts using multi-label structured svm," *BMC Bioinformatics*, vol. 18, pp. 1–11, 2017.
- [12] Qiang, V. H. B. P., C. A. Glas, and T. Vries, "Automated assessment of patients' self-narratives for posttraumatic stress disorder screening using natural language processing and text mining," *Assessment*, vol. 24, pp. 157–172, 2017.
- [13] B. S. S. S., M. Heesacker, S. Benton, and P. Rashidi, "Automatic detection and classification of cognitive distortions in mental health text," pp. 275–280, 2020.
- [14] M. Gaur, "Knowledge-aware assessment of severity of suicide risk for early intervention," pp. 514–525, 2019.
- [15] M. Gaur, V. Aribandi, A. Alambo, U. Kursuncu, K. Thirunarayan, J. Beich, J. Pathak, and A. Sheth, "Characterization of time-variant and time-invariant assessment of suicidality on reddit using c-ssrs," *PLoS one*, vol. 16, no. 5, p. e0250448, 2021.
- [16] S. A. B. F.El, L. Cherrat, A. Moutaouakkil, and M. Ezziyyani, "Deep learning algorithm for suicide sentiment prediction," pp. 261–272, 2018.
- [17] T. and K. T. R., "Predicting mental conditions based on history of present illness in psychiatric notes with deep neural networks," *Journal of Biomedical Informatics*, vol. 75, pp. 138–148, 2017.
- [18] S. and A. G. T., "Depression intensity estimation via social media: A deep learning approach," *IEEE Transactions on Computational Social Systems*, vol. 8, pp. 1465–1474, 2021.
- [19] M. M. L. T. H., B. Xu, and L. Yang, "Detection of suicide ideation in social media forums using deep learning," *Algorithms*, vol. 13, p. 7, 2020.
- [20] A. Moon, "Bangla mental health text classification," GitHub repository, 2025, [Online]. Available: <https://github.com/Asfia-oishy/Mental-Health-Detection>.
- [21] N. Afnan, "Bangla stop words list," <https://www.kaggle.com/datasets/nuhashafnan/bangla-stopwords>, 2022, accessed: February 09, 2024.
- [22] M. I. Protick, "bangla-stemmer (version 1.0)," <https://pypi.org/project/bangla-stemmer/>, 2020, accessed: November 10, 2024.
- [23] A. Wendland, M. Zenere, and J. Niemann, "Introduction to text classification: impact of stemming and comparing tf-idf and count vectorization as feature extraction technique," in *European Conference on Software Process Improvement*. Springer, 2021, pp. 289–300.
- [24] S. learn Developers. (2024) Countvectorizer: Feature extraction for text. <https://tinyurl.com/sklearn-countvectorizer>. Accessed: 2024-11-15.
- [25] —. (2024) Tfidfvectorizer: Term frequency-inverse document frequency transformer. <https://tinyurl.com/sklearn-tfidfvectorizer>. Accessed: 2024-11-15.
- [26] A. Moon, "Custom frequency vectorizer algorithm," GitHub repository, 2025, [Online]. Available: <https://github.com/Asfia-oishy/Mental-Health-Detection/blob/main/CustomizedVectorizer.pdf>.
- [27] —, "Custom frequency vectorizer algorithm," GitHub repository, 2025, [Online]. Available: <https://github.com/Asfia-oishy/Mental-Health-Detection/blob/main/ExplanationWithExample.pdf>.